

Chương này trình bày các công nghệ và nền tảng cơ sở được lựa chọn để giải quyết các bài toán thiết kế trò chơi. Với mỗi công nghệ, ưu nhược điểm so với các lựa chọn thay thế sẽ được phân tích để làm rõ tính hợp lý khi áp dụng vào dự án.

0.1 Nền tảng phát triển Unreal Engine 5.6

Để vận hành một trò chơi 3D góc nhìn thứ ba, yêu cầu về kết xuất đồ họa thời gian thực (Real-time Rendering), mô phỏng vật lý và trí tuệ nhân tạo là rất khắt khe. Dự án quyết định sử dụng bộ khung (Framework) của Unreal Engine 5.6 **UE56Doc**. Phần này tập trung phân tích các thuật toán cốt lõi bên trong Engine giúp giải quyết trực tiếp các yêu cầu kỹ thuật của Đồ án.

0.1.1 Chiều sáng toàn cục với Lumen

Một trong những trọng tâm kỹ thuật của Đồ án là cơ chế cổng không gian. Việc kết nối và di chuyển giữa các khu vực có điều kiện ánh sáng hoàn toàn khác biệt khiến môi trường thị giác phải biến đổi liên tục và linh hoạt.

- **Cơ chế hoạt động:** Lumen là hệ thống Global Illumination (GI) và Reflections động. Thuật toán của Lumen sử dụng hệ thống *Surface Cache* kết hợp cùng *Screen Tracing* và *Software Ray Tracing* dựa trên Signed Distance Fields (SDF) để nội suy ánh sáng theo thời gian thực mà không cần biên dịch trước.
- **Lựa chọn thay thế:**
 - *Kết xuất ánh sáng tĩnh (Light Baking / Lightmap):* Phổ biến và tiêu tốn ít tài nguyên phần cứng, nhưng không đáp ứng được yêu cầu thay đổi ánh sáng thời gian thực khi môi trường bị xáo trộn hoặc khi nhìn qua cổng không gian.
 - *Hardware Ray Tracing truyền thống:* Nội suy tia sáng vật lý chính xác nhưng tiêu tốn tài nguyên khổng lồ, giới hạn đối tượng người chơi ở các dòng phần cứng cao cấp.
- **Lý do lựa chọn:** Lumen giải quyết trọn vẹn bài toán chiều sáng cho hệ thống ống kính cổng không gian, đồng thời tối ưu hóa tốc độ khung hình (Framerate) bằng cách giảm thiểu nội suy tia sáng qua Global Distance Field và tận dụng hiệu quả bộ nhớ đệm (Cache).

0.1.2 Trí tuệ nhân tạo qua Cây hành vi (Behavior Tree)

Hệ thống kẻ địch (Guard) yêu cầu khả năng xử lý hàng loạt hành vi phức tạp như: Tuần tra tự do, Điều tra âm thanh khả nghi, Ẩn nấp và Giao tranh khi phát hiện người chơi.

- **Cơ chế hoạt động:** Cây hành vi phân nhánh quy trình ra quyết định từ gốc

(Root) qua các nút kiểm soát (Sequence, Selector). Việc kết hợp với bảng dữ liệu bộ nhớ chia sẻ (Blackboard) cho phép AI đánh giá biến số môi trường để chọn ra nhánh logic thực thi phù hợp nhất.

- **Lựa chọn thay thế - Máy trạng thái (State Machine):** Rất dễ triển khai cho AI cơ bản (chỉ gồm trạng thái Đứng yên và Tấn công). Tuy nhiên, khi hệ thống hành vi mở rộng thêm các nhánh Tuần tra, Phỏng đoán (Overwatch) hay Ẩn nấp (Cover), Máy trạng thái sẽ nhanh chóng trở thành một mạng lưới logic chằng chịt, khó gỡ lỗi và bảo trì (Spaghetti Anti-pattern).
- **Lý do lựa chọn:** Đặc tính mô-đun hóa cao của Cây hành vi là thuật toán lý tưởng nhất. Nhờ đó, dự án có thể dễ dàng thiết kế các tập tính độc lập cho kẻ thù (như Điều tra tích cực, Cố thủ, hoặc Tấn công chiến thuật) thông qua việc tái sử dụng các tác vụ (Tasks) đã được lập trình sẵn.

0.2 Hệ thống kiểm soát phiên bản Perforce Helix Core (P4V)

Với đặc thù của kiến trúc trò chơi 3D, toàn bộ tài nguyên (Texture, Audio) và bản đồ màn chơi (.umap) thường xuyên được lưu trữ ở định dạng nhị phân nguyên khối (Binary files) với dung lượng từ vài trăm Megabytes đến hàng Gigabytes. Hệ thống phiên bản Perforce Helix Core (P4V) **Perforce** được dùng nhờ yêu cầu khả năng làm việc đa người dùng mà không làm phá vỡ tính toàn vẹn của mã nguồn.

0.2.1 Các nền tảng thay thế

- **Git (kết hợp Git LFS):** Vô cùng mạnh mẽ và linh hoạt trong việc quản lý cây mã nguồn phân tán cho tệp văn bản (C++, Python...). Tuy nhiên, nếu hai thành viên cùng chỉnh sửa một tệp bản đồ 3D nhị phân, Git sẽ không thể hợp nhất (Merge) kết quả. Việc giải quyết xung đột (Conflict) trên Git LFS đối với file nhị phân thường gây gián đoạn nghiêm trọng tiến trình phát triển.
- **Subversion (SVN):** Quản trị dữ liệu tập trung nhưng tốc độ giao tiếp và gửi tệp bị giới hạn. Năng lực xử lý hệ sinh thái nhị phân 3D chậm và vẫn tiềm ẩn rủi ro xung đột cục bộ.

0.2.2 Lý do sử dụng Perforce (P4V)

- **Cơ chế quản trị tập trung và Khóa tệp (File Locking):** Bất cứ khi nào một mô hình hoặc màn chơi cần sửa đổi, giao thức Checkout của P4V sẽ lập tức khóa tệp đó ở trạng thái "Chỉ đọc" (Read-Only) đối với tất cả các thành viên khác trong mạng lưới. Giải pháp này loại bỏ hoàn toàn rủi ro ghi đè và xung đột tệp nhị phân.
- **Tích hợp hệ sinh thái nội tại:** P4V cho phép đẩy dữ liệu (Submit) mạnh mẽ qua các tệp Workspace. Khả năng liên kết trực tiếp vào giao diện người

dùng của Unreal Engine hỗ trợ quá trình lưu trữ lịch sử minh bạch, cho phép khôi phục (Rollback) nhanh chóng các tệp tài nguyên ở bất kỳ giai đoạn thực nghiệm nào của Đồ án.